Searching for Numerical Models of Chemical and Biological Dynamics

Dennis George Wilson DENNISWILSON@SAKANA.AI
Sakana AI
Tokyo, Japan

Artem Shmatko ARTEMSHMATKO@SAKANA.AI
Sakana AI
Tokyo, Japan

Stefania Druga STEFANIA@SAKANA.AI
Sakana AI
Tokyo, Japan

Abstract

Modeling and simulation are central to scientific discovery. Digital twins and *in silico* experiments enable faster hypothesis testing, cheaper experiments, and experiments on systems that cannot be manipulated directly. Where *ab initio* simulation is prohibitively expensive, or the rules governing a system are too complex to formalize by hand, neural networks are commonly used in place of an explicit simulator. This removes the need to write the simulator by hand, but the result is a black box. The underlying mechanism cannot be inspected, limiting validation of scientific correctness and possible insight. We close this gap with AutoSim, a programmatic regression system in which coding agents write human-readable numerical models fitted to observed data. We evaluate it across three chemical and biological systems of increasing complexity: simulated Gray–Scott reaction–diffusion, a continuum active matter model, and real-world microscopy imaging of self-organizing behavior in oil droplets. On simulated systems, AutoSim-designed programs reach the lowest error among neural, symbolic, and generative baselines, while additionally providing inspection-ready code artifacts. For oil droplets, where the driving fields are difficult to observe, no method accurately simulates the motion. However, the AutoSim program models droplets as objects rather than pixels and shows realistic movement. We propose this automatic process for numerical simulation development as a new tool for insights into biological and chemical processes.

1. Introduction

The development of a numerical model of a chemical or biological process goes through many steps before it yields a simulation that can be used to understand and probe the process. An essential design choice is the scope of the model: which mechanisms to include and which assumptions to make. For example, spatial transcriptomics data may or may not warrant a term for lateral diffusion of transcripts; an active suspension may or may not require hydrodynamic coupling. Each added component introduces degrees of freedom that the model must be tested over, and unless its parameters and priors have been carefully tuned, it is impossible to say how the new version compares to its predecessor. When a model version fits the data poorly, the cause may lie in its core mechanism or an insufficiently explored implementation, including hyperparameter tuning, and the two are difficult to tease apart.

© 2026 D.G. Wilson, A. Shmatko & S. Druga.

As a result, a single iteration of model development routinely takes a researcher weeks of time. These delays are well documented: scientists spend 30% or more of their time on software (Prabhu et al., 2011; Wilson et al., 2014), and tuning a large model against observations takes months to years (Hourdin et al., 2017).

One possible intervention is to automate the construction of the simulator. Neural surrogates learn dynamics directly from trajectories (Sanchez-Gonzalez et al., 2020; Li et al., 2021) and can be both fast and accurate, but are inherently black boxes that expose no mechanism for scientists to read and no assumption to test. Physics-informed neural networks (Cuomo et al., 2022) constrain the learned solution with a known governing equation, but the equation must be supplied in advance. Symbolic regression produces interpretable closed-form equations (Brunton et al., 2016; Cranmer, 2023), but is limited to a predefined set of operators. It also stops at the equation, so the discretization schemes and solvers that turn it into a running simulation must still be built separately.

Recent program-search methods, such as FunSearch (Romera-Paredes et al., 2024) and ShinkaEvolve (Lange et al., 2025), make a third option practical: candidate models are complete, executable simulators generated and revised by language models (Li et al., 2026). We argue that this is the natural form for model discovery to take, because a program specifies its state variables, operators, fitted parameters and integration schemes explicitly, so every modeling assumption can be located in the source, inspected and modified. We call this **programmatic regression**, a form of program synthesis with the same data-driven goal as symbolic regression, and present AutoSim, an agentic method that performs it by iteratively refining executable code against the data. This mirrors the propose–implement–fit–inspect–revise loop that modelers otherwise perform manually, accelerating it while remaining transparent to the researcher.

We evaluate AutoSim on three chemical and biological systems of increasing complexity: a simulation whose governing equations are known (Gray–Scott reaction–diffusion (Pearson, 1993)), a simulation in which only part of the state is observed (continuum active matter (Maddu et al., 2024)), and real microscopy of self-organizing oil droplets (Liu et al., 2024b), in which the fields that drive the motion are not recorded. Search using AutoSim arrives at different kind of models for different systems: a spectral PDE solver for Gray–Scott, a description of individual droplets rather than of pixels for the oil droplets dataset. This demonstrates that programmatic regression is not restricted to a fixed representation but is open over the full space of possible simulation programs.

2. Related work

Neural surrogates. Neural surrogates approximate the update rule of a simulation using a trainable neural network. On gridded fields the standard architectures are convolutional U-Nets and ConvNeXt backbones (Ronneberger et al., 2015; Liu et al., 2022); Fourier Neural Operators learn a resolution-invariant operator between function spaces (Li et al., 2021). Meta-learning, as in DISCO (Morel et al., 2025), trains a single model over a diverse set of physical problems, improving generalization and data efficiency on downstream tasks. A forecast from a neural surrogate is typically cheaper than the underlying physics-based simulation, which motivates their use in climate science and weather forecasting (Bi et al., 2023; Lam et al., 2023). However, due to their large number of parameters, neural surrogates

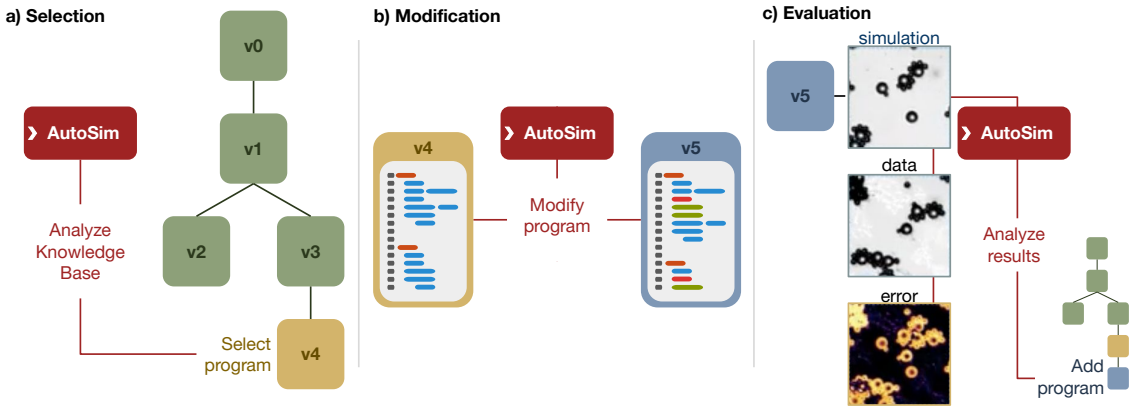


Figure 1: AutoSim develops a simulation program through repeated selection, modification, and evaluation. (a) Separate agent calls inspect the Knowledge Base to select a version of the program, (b) modify the selected code to make a new candidate, and (c) evaluate the data generated by the new candidate against observational data. Admitted programs and analyses are added to the Knowledge Base for later iterations.

are black-box estimators. The learned rule exposes no mechanism to inspect and no modeling assumptions to test, which limits its use for scientific discovery.

Equation discovery. An alternative, interpretable method is to search for equations. Methods like SINDy use sparse regression over a library of candidate terms to recover ordinary and partial differential equations from data (Brunton et al., 2016; Rudy et al., 2017). Recent work has extended these methods to noisy and scarce measurements (Both et al., 2021; Chen et al., 2021) and replaces a fixed library with reinforcement-learning search over open-form expressions (Du et al., 2024). Symbolic regression searches expression trees directly (Udrescu and Tegmark, 2020; Cranmer, 2023) and has been used to model physical systems (Al Najar et al., 2026; AbdusSalam et al., 2025). In chemistry and biology, symbolic regression has been used to recover reaction networks from concentration data (Hoffmann et al., 2019) and interaction laws for particles from trajectories (Lu et al., 2019; Cranmer et al., 2020). Recent work uses a language model to propose candidate expressions (Shojaee et al., 2025a). However, these methods rely on hand-written numerical infrastructure, integration schemes and solvers that lie outside the search for equations. These choices require extensive implementation effort and constrain the space of possible solutions.

Programmatic regression. Removing the hand-written infrastructure altogether leads to a class of methods in which a large language model (LLM) writes the entire simulation as executable source code. FunSearch and AlphaEvolve evolve programs against a scalar score by repeatedly mutating an archive of parents, with results on mathematical and algorithmic problems (Romera-Paredes et al., 2024; Novikov et al., 2025). ShinkaEvolve makes the same search sample-efficient through adaptive parent selection and novelty-based rejection of near-duplicate candidates (Lange et al., 2025). AutoSim operates in a similar framework but

adapts the methodology for numerical simulations and the use of coding agents. The closest prior work is the Scientific Generative Agent (Ma et al., 2024), in which an LLM proposes a discrete hypothesis as code and a differentiable simulator fits its continuous parameters. AutoSim differs in searching over the whole simulator, numerics included, in keeping every admitted program in a branching archive with shared observations, and in evaluating under a trusted process that agents cannot reach.

3. Programmatic regression with AutoSim

Programmatic regression searches for a program p that generates data $\hat{x}$, which is compared to data x, for example by calculating the error between them. AutoSim searches specifically for programs that simulate a physical process. Given observations $x_{0:k}$ up to time k, metadata m, and a prediction horizon T, a candidate program p implements

$$p(x_{0:k}, m) \longrightarrow \hat{x}_{k+1:k+T}, \tag{1}$$

that is, it reads an observed window and returns the next T frames of the simulation. AutoSim uses an iterative search method to find programs which generate data that corresponds to the target data. No specific implementation details of the program are imposed, apart from the use of a supported language. AutoSim supports Python and Julia (Bezanson et al., 2017); all experiments in this study use Julia for computational efficiency. All other program implementation choices, such as discretization schemes, library use, and the general modeling methodology, are made during the AutoSim search.

The search method of AutoSim is shown in Figure 1, and an example on Gray–Scott is given in Figure 2. In a growing Knowledge Base, AutoSim maintains a branching tree of programs for each dataset and uses agents to refine it iteratively against a predefined set of metrics. At each iteration, a selection agent reads the tree and the accumulated evidence in the Knowledge Base and chooses a parent program. A modification agent diagnoses the selected parent and writes a new candidate program. A trusted evaluation process executes the candidate and computes all metrics against held-out targets that agents cannot access. Finally, an evaluation agent summarizes the resulting behavior for later selection calls. The agents are general-purpose LLMs using coding harnesses; their parameters are not updated during the search. This search method runs for a fixed number of iterations or until a predefined score is reached.

The Knowledge Base of AutoSim serves as the shared memory of the search. It is a typed store, written only by the control process, that holds the dataset description and any problem guidance, every program with its parent, source, execution status, and scores, the trusted measurements of its behavior, and the analyses the agents wrote about it. The Knowledge Base is a SQLite database, and AutoSim provides a set of command line tools to facilitate tracking program versions and evaluation metrics specific to the Knowledge Base's structure.

An important aspect of AutoSim is the definition of metrics used to evaluate a simulation. These metrics are provided to the coding agents to guide the selection, modification, and evaluation steps of AutoSim. Coding agents optimize based on the available information, and optimizing these metrics can carry them outside the scope of the task. Reward hacking, reading or modifying the grader, retrieving answers from disk or the internet, or breaking

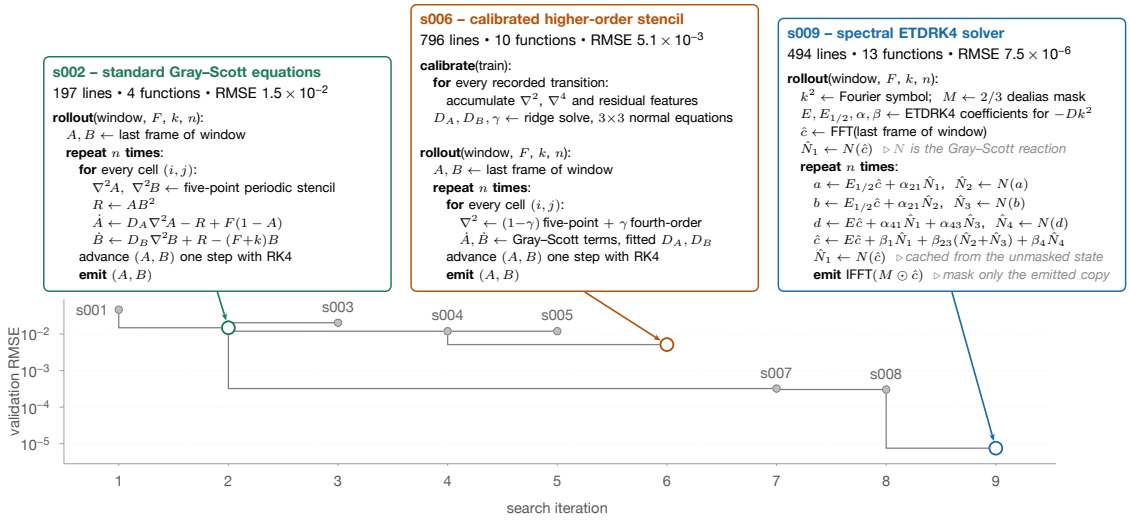


Figure 2: One AutoSim search on Gray–Scott. Each node is a candidate program from an example search process. Three candidates are shown in pseudocode. `s002` writes the Gray–Scott equations directly. `s006` keeps finite differences and calibrates two diffusivity scales and a stencil blend against the training transitions; the search abandoned this branch. `s009` moves to a Fourier representation with a Cox–Matthews ETDRK4 update. The search minimizes validation RMSE while composing increasingly complex programs; all program versions remain interpretable for post-hoc analysis.

out of the sandbox altogether are possible (Greenblatt et al., 2026). In AutoSim, data, execution, and scoring therefore stay in a trusted process the agents cannot access. Each evaluation is performed in a sandbox without network access, checks that the output is finite and well-formed within a runtime limit, and returns only validation scores, minimizing data leak risk.

We find that AutoSim is able to generate physically meaningful programs for the three tasks in this study, as exemplified in Figure 2. The specific metrics provided, notably RMSE, orient the search towards overly detailed or engineered solutions. While AutoSim is an automatic process, its design and focus on interpretable results is intended as a human-facing method. The ultimate analysis and choice of the resulting programs is performed by a human expert.

4. Results

We evaluate AutoSim on three systems of increasing complexity: Gray–Scott reaction–diffusion (Pearson, 1993; Ohana et al., 2024), whose governing equations are known; a continuum active matter model (Maddu et al., 2024), which releases only low-order moments of the state its source simulator integrates; and microscopy of self-organizing oil droplets (Liu

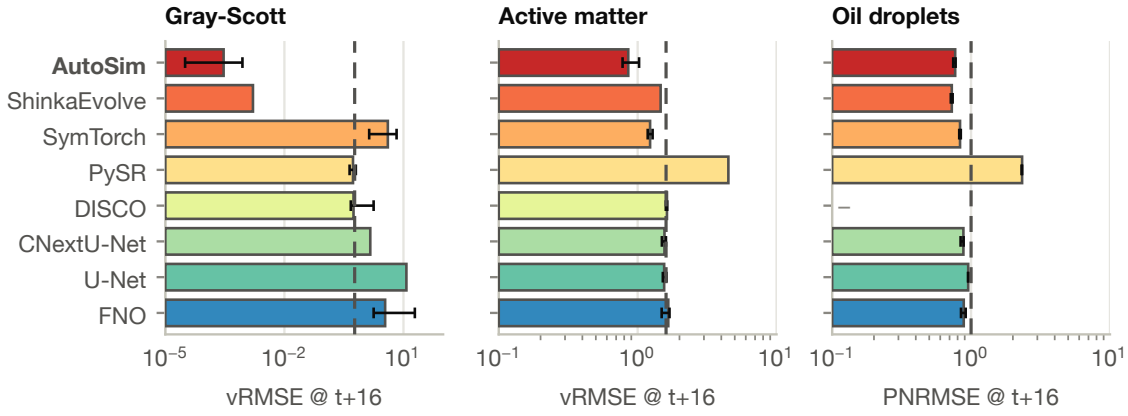


Figure 3: Rollout error at t+16 on the three datasets (lower is better; logarithmic horizontal scale). Bars are the reported method-level medians over runs; error bars span the minimum to the maximum over runs. The dashed line is the persistence control; DISCO was not run on the oil droplets task.

et al., 2024a), which does not record the fields that drive the motion. section A describes the three datasets.

4.1. Established chemical and biological simulations

On Gray–Scott and active matter we compare AutoSim with ShinkaEvolve (Lange et al., 2025), SymTorch (Tan et al., 2026), PySR (Cranmer, 2023), DISCO (Morel et al., 2025), a CNextU-Net (Liu et al., 2022), a conventional U-Net (Ronneberger et al., 2015), an FNO (Li et al., 2021), and persistence. For rollout i, horizon t, and field c, let e_{itc} be spatial RMSE and let $B_{itc} = \sqrt{\mathrm{Var}(x_{itc}) + 10^{-7}}$.

We report the field-balanced variance-normalized error (vRMSE)

$$\mathrm{vRMSE}_{it} = \frac{1}{C} \sum_{c=1}^{C} \frac{e_{itc}}{B_{itc}}. \tag{2}$$

Table 1 reports the vRMSE across all methods and multiple horizons; values for the $t+16$ horizon are shown in Figure 3. Values are medians over four AutoSim runs on Gray–Scott and active matter, three on the oil droplets, and three runs for each baseline. The terminal horizons are 995 steps for Gray–Scott and 75 for active matter.

AutoSim achieves lower error than other methods on both Gray–Scott and Active matter tasks, across rollout horizons. On Gray–Scott, AutoSim reaches 2.41×10^{-4} vRMSE at $t+4$ and 4.91×10^{-3} at $t+995$, compared with 1.61×10^{-3} and 2.88×10^{-2} for ShinkaEvolve. These are 6.7- and 5.9-fold reductions, respectively. On active matter, AutoSim has the lowest vRMSE at every horizon in Table 1: 0.345, 0.856 and 1.44, approximately 39%, 30% and 17% lower than the strongest alternative at each horizon. This demonstrates that programmatic regression can create simulation data that accurately matches target data.

Table 1: Autoregressive rollout error (lower is better). Gray–Scott and active matter report vRMSE at t+4, t+16, and the terminal horizon; oil droplets report PN-RMSE over 72 test windows at t+4 and t+16. Bold marks the lowest value in each column. Dashes denote unavailable values: DISCO was not run on the oil droplets task, and PySR diverged early on the Active matter task.

	Gray–Scott			Active matter			Oil droplets	
Method	t+4	t+16	t+995	t+4	t+16	t+75	t+4	t+16
AutoSim	**0.000241**	**0.000294**	**0.00491**	**0.345**	**0.856**	**1.44**	0.751	0.768
ShinkaEvolve	0.00161	0.00161	0.0288	0.811	1.46	2.08	**0.621**	**0.726**
SymTorch	1.39	4.04	24.7	1.12	1.23	3.43	0.958	0.833
PySR	0.185	0.535	1.40	2.51	4.50	—	1.62	2.33
DISCO	0.121	0.551	5.00	0.945	1.63	5.40	—	—
CNextU-Net	0.511	1.46	4.18	0.568	1.56	1.73	0.732	0.881
U-Net	15.9	11.8	113	1.36	1.55	1.78	0.792	0.954
FNO	0.970	3.45	37.0	1.07	1.66	1.99	0.961	0.886
Persistence	0.212	0.589	0.844	1.02	1.60	2.44	1.00	1.00

Figure 4 visualizes the error of each method at t+16. For both AutoSim and ShinkaEvolve, the error on Gray–Scott is negligible. Notably, the FNO produced non-realistic error signatures. In the Active matter problem, distinguishing realistic vs non-realistic is difficult due to the high stochasticity of the data; however, upon visual inspection of the generated videos, we note that the error patterns of AutoSim match the target data's form more than the other methods'.

4.1.1. AutoSim search analysis

In Figure 2, we show the AutoSim search tree for the Gray–Scott task and an inspection of three candidate simulations. These programs follow increasing levels of complexity, from the canonical reaction terms to a calibrated data fit, and finally to a spectral ETDRK4 solver. The full code of every simulation in every run is provided in the accompanying repository[1]. The run displayed in Figure 2 is Gray–Scott `run00`.

Search methods begin the same way for Gray–Scott and active matter. In all eight runs, the very first candidate solves no PDE, but every run writes the governing equations by its second or third iteration. What follows differs. On Gray–Scott, writing the equations with a finite-difference Laplacian and RK4 brings all four runs to the same error, and the remaining two to three orders of magnitude come from numerics: a Fourier representation, an exponential time-differencing integrator (Cox and Matthews, 2002; Kassam and Trefethen, 2005), and the order in which the dealias filter and the cached nonlinearity are applied within a substep, which alone accounts for a factor of 40. Two of the four runs reached the stop criterion early and returned different programs, of 494 and 534 lines, whose validation errors agree to 14 significant figures.

1. https://anonymous.4open.science/r/simbiochem-results-C6FD/

On active matter, the first candidate that solves a PDE is already the full active-nematic system, and the search then stops improving: that candidate captures most of each run's total gain, and the twenty or more iterations after it vary the closure without improving on it by more than 10%. Where the data determine the mechanism the search finds it and spends the rest of its budget on numerical accuracy; where they do not, it fixes the model class immediately and the remaining iterations cannot resolve what the observations leave open.

4.2. Oil droplets

On the droplets task, we run the same methods as in subsection 4.1, with neural predictors forecasting all 16 future grayscale fields autoregressively. We omit the DISCO method (Morel et al., 2025) as its pretraining and implementation, which are suitable for simulations as in Ohana et al. (2024), are not readily adaptable to the oil droplets dataset.

For this dataset, persisting the first frame over time scores well due to the complexity of the task. We therefore use, as the primary score, a persistence-normalized RMSE (PN-RMSE):

$$E_{\text{drop}} = \frac{1}{3}\sum_{q=1}^{3}\frac{1}{|\mathcal{M}_q|}\sum_{i\in\mathcal{M}_q}\sqrt{\frac{\sum_{w,t,\mathbf{s}} m(\mathbf{s})[\hat{x}_{iwt}(\mathbf{s}) - x_{iwt}(\mathbf{s})]^2}{\sum_{w,t,\mathbf{s}} m(\mathbf{s})[x_{iwt}(\mathbf{s}) - x_{iw0}(\mathbf{s})]^2}}. \tag{3}$$

Here $\mathcal{M}_q$ is the set of test movies in diameter-ratio regime q, w indexes the twelve evaluation windows of movie i, $m(\mathbf{s})$ the fixed center mask, and x_{iw0} the last observed field, so persistence scores exactly 1.

Six of the seven methods score below the persistence control at both horizons, and they fall in a narrow band: at t+16 every method except PySR lies between 0.726 and 0.954. ShinkaEvolve has the lowest value in both droplet columns, with AutoSim next at t+16; ShinkaEvolve alone ran without the scoring mask and the condition planes the other methods received. Across the two horizons AutoSim is the method whose ratio against persistence changes least, from 0.751 to 0.768.

The numerically similar metric scores, however, mask the massive variance in the structure of the solutions (Figure 4). The droplets are sparse and dark on a bright field, so a forecast that removes them scores well. For example, the FNO forecast returns a near-empty field on the movies with few droplets. Returning the background is not by itself sufficient to score well across all data, but PN-RMSE cannot distinguish a forecast that places droplets slightly wrong from one that does not place them at all. This is a limitation of informing the analysis based on a single metric, which must compress complex information about droplet movement, shape, and interactions into a single value.

Rather than evaluate simply based on error compared to video footage, we can also evaluate if methods reproduce results in a realistic way. All methods except AutoSim exhibit visual warping of the droplet form and diffusion of the droplets into their surrounding liquid, as shown in section B. We posit that this non-realistic simulation effect is due to error minimization; moving the droplets, even through deformation, in the direction of the fluid flow reduces error.

AutoSim allows for inspection of models beyond their output data and metrics, however. We note that selected programs use an object-based model focused on droplets. Droplets are detected in the observed window, tracked, and their centers integrated. The forecast is rendered by placing each droplet's contrast at its integrated position. This representation

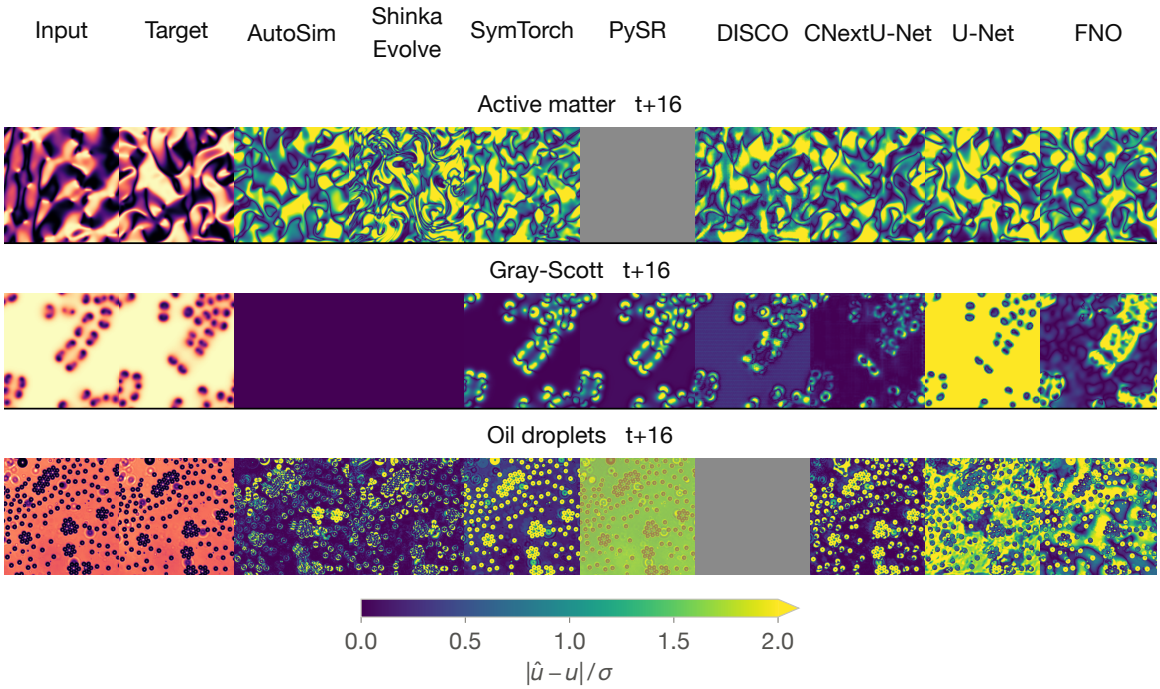


Figure 4: Rollout error at t+16 on the three datasets. Input is the observed frame; Target is the ground truth at t+16. Each method panel shows |prediction − truth| divided by the target's spatial standard deviation. Grey panels indicate no run for DISCO on active droplets and divergence for PySR on active matter. Error for AutoSim and ShinkaEvolve on Gray–Scott is not visible at this scale.

appeared even though no representation was prescribed by the task. We posit that AutoSim can be used to search for, confirm, or discover appropriate modeling representations for expert analysis.

5. Discussion

The three tasks differ not only in nature and complexity, but also in the class of challenge AutoSim has to overcome to produce a faithful simulation. For Gray–Scott the general form of the solution is known, so the search concentrates on finding a suitable numerical implementation. Importantly, the candidates are evaluated not against a platonic ground-truth but against the output of another numerical simulation. Beyond the underlying chemical processes, AutoSim therefore also has to reproduce the numerical signature of the reference implementation, such as discretization, time step, and integrator, which closely resembles fitting the instrument-induced systematic biases of real data.

The active matter task adds another level of complexity. The evolution of the system depends on the particle orientation distribution, a hidden variable, so the search has to choose a closure, an assumed relation between the observed moments and the unobserved

ones, and revise it as evidence accumulates. Finally, the droplet task shows how multilayered a scientific modeling problem can be. Unlike the other tasks, the simulation is object-centric, with individual droplets acting as cohesive physical bodies rather than as pixels. Furthermore, the objects interact with each other and with the surrounding fluid flow, which has to be modeled separately.

The complexity of the task rises with the number of layers of abstraction, and the performance of AutoSim varies accordingly. While it achieves major gains over competing methods on Gray–Scott and active matter, it does not capture all components of the droplet system and cannot predict its behavior reliably. The difficulty that LLM agents have with long-range independent research is well documented (Kwa et al., 2025; Chen et al., 2025). This task highlights the importance of initializing the Knowledge Base with the known properties and invariants of the system, and of metrics that not only quantify forecasting error but also track conservation laws and other structural constraints. Further generations of such systems can also be built around a human-in-the-loop model, in which the researcher guides the agentic system through particularly complex tasks.

Besides lacking a researcher's prior intuition about the task, LLMs are also limited in their ability to explore it. The data of a simulation are inherently spatiotemporal, but without native video input, the agents judge them through summary statistics and sampled frames, and may arrive at biased conclusions about the dynamics. Closing this gap with models that take video as native input is a natural next step for AutoSim.

6. Limitations

In AutoSim, the metrics used to evaluate solutions are chosen manually and the agents pursue them, including through solutions that satisfy metrics without capturing the dynamics. A smoothed field can have low pixel error while every object in it is misplaced. The trusted evaluator in AutoSim prevents cheating on the system, but not on the metric. The pressure is greatest on systems that are complex or strongly stochastic. Where genuine prediction is hardest, such local optima are the most difficult to escape.

Second, the cost of AutoSim search scales with the cost of the simulation itself. Every hypothesis has to be executed over several trajectories before it can be scored, so where a single simulation is already expensive, a search over many hypotheses further increases the cost.

Finally, while data leaks in the usual sense are prevented by the system design, "concept leaks" are not. The agents are familiar with the standard model systems, Gray–Scott among them, and rely on that knowledge when writing candidate solutions. This confound is documented for equation discovery with language models, where benchmarks built on textbook equations are inflated by memorization (Shojaee et al., 2025b). Complex real-world systems with *a priori* unknown behavior should always be used in evaluation alongside the synthetic tasks.

7. Conclusion

Executable programs are a representation in which modeling assumptions can be located, tested, and revised, and program search with language models makes exploring them practical.

On Gray–Scott, continuum active matter, and droplet microscopy, AutoSim finds programs whose long-horizon error is the lowest among the compared methods or close to it, while returning a model that is transparent and interpretable. Beyond the selected program for each task, the tree of all generated programs is a research artifact in its own right. The AutoSim Knowledge Base stores the competing hypotheses of a search and allows them to be compared quantitatively. Where several are equally supported by the data, a working simulation of each offers a starting point for deciding which experiments to conduct next. Much remains to be done, particularly in designing systems that work under the fuzzy constraints of "good science", but the flexibility of entire programs written by LLM agents, together with the control that their transparency leaves with the researcher, makes programmatic regression a promising route to assisting scientific discovery.

Data availability

The Gray–Scott and active matter trajectories are publicly available through The Well (Ohana et al., 2024). The oil droplet videos and species-specific tracks are available under CC BY 4.0 at the versioned DOI 10.5281/zenodo.11180025 (Liu et al., 2024a). We will release AutoSim as well as the results, preprocessing contract, complete-movie split manifest, plotting data, and evaluation configuration upon publication. AutoSim-generated programs are available here: https://anonymous.4open.science/r/simbiochem-results-C6FD/.

References

Shehu AbdusSalam, Steven Abel, and Miguel Crispim Romão. Symbolic regression for beyond the standard model physics. *Physical Review D*, 111(1):015022, 2025.

Mahmoud Al Najar, Dennis G Wilson, and Rafael Almar. Interpretable machine learning for shoreline forecasting. *Scientific Reports*, 16(1):11457, 2026.

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi: 10.1137/141000671.

Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Accurate medium-range global weather forecasts with 3D neural networks. *Nature*, 619:533–538, 2023. doi: 10.1038/s41586-023-06185-3.

Gert-Jan Both, Subham Choudhury, Pierre Sens, and Remy Kusters. DeepMoD: Deep learning for model discovery in noisy data. *Journal of Computational Physics*, 428:109985, 2021. doi: 10.1016/j.jcp.2020.109985.

Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. doi: 10.1073/pnas.1517384113.

Zhao Chen, Yang Liu, and Hao Sun. Physics-informed learning of governing equations from scarce data. *Nature Communications*, 12:6136, 2021. doi: 10.1038/s41467-021-26434-1.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, 2025.

Steven M. Cox and Paul C. Matthews. Exponential time differencing for stiff systems. *Journal of Computational Physics*, 176(2):430–455, 2002. doi: 10.1006/jcph.2002.6995.

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023. doi: 10.48550/arXiv.2305.01582.

Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho. Discovering symbolic models from deep learning with inductive biases. In *Advances in Neural Information Processing Systems*, volume 33, pages 17429–17442, 2020. arXiv:2006.11287.

Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what's next. *Journal of Scientific Computing*, 92(3):88, 2022. doi: 10.1007/s10915-022-01939-z.

Mengge Du, Yuntian Chen, and Dong-juan Zhang. Discover: Deep identification of symbolically concise open-form partial differential equations via enhanced reinforcement learning. *Physical Review Research*, 6, Feb 2024. ISSN 2643-1564. doi: 10.1103/physrevresearch.6.013182. URL https://doi.org/10.1103/physrevresearch.6.013182.

Ryan Greenblatt, Ajeya Cotra, and Hjalmar Wijk. Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. METR and Redwood Research, https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/, 2026. August 26.

Moritz Hoffmann, Christoph Fröhner, and Frank Noé. Reactive SINDy: Discovering governing reactions from concentration data. *The Journal of Chemical Physics*, 150(2):025101, 2019. doi: 10.1063/1.5066099.

Frédéric Hourdin, Thorsten Mauritsen, Andrew Gettelman, Jean-Christophe Golaz, Venkatramani Balaji, Qingyun Duan, Doris Folini, Duoying Ji, Daniel Klocke, Yun Qian, Florian Rauser, Catherine Rio, Lorenzo Tomassini, Masahiro Watanabe, and Daniel Williamson. The art and science of climate model tuning. *Bulletin of the American Meteorological Society*, 98(3):589–602, 2017. doi: 10.1175/BAMS-D-15-00135.1.

Aly-Khan Kassam and Lloyd N. Trefethen. Fourth-order time-stepping for stiff PDEs. *SIAM Journal on Scientific Computing*, 26(4):1214–1233, 2005. doi: 10.1137/S1064827502410633.

Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, Ryan Bloom, Thomas Broadley, Haoxing Du, Brian Goodrich, Nikola Jurkovic, Luke Harold Miles, Seraphina Nix, Tao Lin, Neev Parikh, David Rein, Lucas Jun Koba Sato, Hjalmar Wijk, Daniel M. Ziegler, Elizabeth Barnes, and Lawrence Chan. Measuring AI ability to complete long tasks, 2025. arXiv:2503.14499.

Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, and Peter Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023. doi: 10.1126/science.adi2336.

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025. doi: 10.48550/arXiv.2509.19349.

Shanda Li, Tanya Marwah, Junhong Shen, Weiwei Sun, Andrej Risteski, Yiming Yang, and Ameet Talwalkar. CodePDE: An inference framework for LLM-driven PDE solver generation. *Transactions on Machine Learning Research*, 2026. arXiv:2505.08783.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

Yutong Liu, Ramalingam Kailasham, Pepijn G. Moerman, Aditya S. Khair, and Lauren D. Zarzar. Self-organized patterns in predator-prey droplets, 2024a. Versioned dataset record, CC BY 4.0.

Yutong Liu, Ramalingam Kailasham, Pepijn G. Moerman, Aditya S. Khair, and Lauren D. Zarzar. Self-organized patterns in non-reciprocal active droplet systems. *Angewandte Chemie International Edition*, 63(49):e202409382, 2024b. doi: 10.1002/anie.202409382.

Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11976–11986, 2022. doi: 10.1109/CVPR52688.2022.01167.

Fei Lu, Ming Zhong, Sui Tang, and Mauro Maggioni. Nonparametric inference of interaction laws in systems of agents from trajectory data. *Proceedings of the National Academy of Sciences*, 116(29):14424–14433, 2019. doi: 10.1073/pnas.1822012116.

Pingchuan Ma, Tsun-Hsuan Wang, Minghao Guo, Zhiqing Sun, Joshua B. Tenenbaum, Daniela Rus, Chuang Gan, and Wojciech Matusik. LLM and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 33940–33962, 2024. arXiv:2405.09783.

Suryanarayana Maddu, Scott Weady, and Michael J. Shelley. Learning fast, accurate, and stable closures of a kinetic theory of an active fluid. *Journal of Computational Physics*, 504:112869, 2024. doi: 10.1016/j.jcp.2024.112869.

Rudy Morel, Jiequn Han, and Edouard Oyallon. DISCO: Learning to discover an evolution operator for multi-physics-agnostic prediction. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. arXiv:2506.13131.

Ruben Ohana, Michael McCabe, Lucas Meyer, Rudy Morel, et al. The well: A large-scale collection of diverse physics simulations for machine learning. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. arXiv:2412.00568.

John E. Pearson. Complex patterns in a simple system. *Science*, 261(5118):189–192, 1993. doi: 10.1126/science.261.5118.189.

Prakash Prabhu, Thomas B. Jablin, Arun Raman, Yun Zhang, Jialu Huang, Hanjun Kim, Nick P. Johnson, Feng Liu, Soumyadeep Ghosh, Stephen Beard, Taewook Oh, Matthew Zoufaly, David Walker, and David I. August. A survey of the practice of computational science. In *State of the Practice Reports, SC '11: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2011.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241, 2015. doi: 10.1007/978-3-319-24574-4_28.

Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations. *Science Advances*, 3(4):e1602614, 2017. doi: 10.1126/sciadv.1602614.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8459–8468. PMLR, 2020.

Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan Reddy. LLM-SR: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations*, 2025a.

Parshin Shojaee, Ngoc-Hieu Nguyen, Kazem Meidani, Amir Barati Farimani, Khoa D. Doan, and Chandan K. Reddy. LLM-SRBench: A new benchmark for scientific equation discovery with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025b. arXiv:2504.10415.

Elizabeth SZ Tan, Adil Soubki, and Miles Cranmer. Symtorch: A framework for symbolic distillation of deep neural networks. In *Catch, Adapt, and Operate: Monitoring ML Models Under Drift Workshop*, 2026.

Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science advances*, 6(16):eaay2631, 2020.

Greg Wilson, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Kathryn D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1):e1001745, 2014. doi: 10.1371/journal.pbio.1001745.

Appendix A. Chemical and biological tasks

We evaluate AutoSim on three datasets from chemistry and biology representing different modelling challenges. Gray–Scott (Pearson, 1993; Ohana et al., 2024) is a positive control whose equations are known, so it tests whether the search assembles a correct and stable solver. Active matter (Maddu et al., 2024) releases only low-order moments of the integrated state. The droplet microscopy (Liu et al., 2024a) does not record the fields that drive the motion, so the search has to choose its own representation.

A.1. Gray–Scott reaction–diffusion

The Gray–Scott model describes two reacting and diffusing species on a periodic domain:

$$\partial_t A = d_A \nabla^2 A - AB^2 + f(1 - A), \tag{4}$$

$$\partial_t B = d_B \nabla^2 B + AB^2 - (f + k)B \tag{5}$$

(Pearson, 1993). The Well release contains 1,200 trajectories on a 128×128 grid with 1,001 stored times, across six feed/kill regimes that produce gliders, bubbles, mazes, worms, spirals, and spots (Ohana et al., 2024). The source data were generated with a Fourier spectral discretization and fourth-order exponential time differencing. Some regimes become nearly stationary, so a model that copies its last input scores well at short horizons, and we report persistence as a control. The task tests whether the search assembles a correct solver for a known closed system and integrates it stably far beyond the validation horizon.

A.2. Continuum active matter

The active-matter task simulates rod-like active particles suspended in a Stokes fluid (Maddu et al., 2024). Physically, the rods stand for self-propelled swimmers such as bacteria or for motor-driven cytoskeletal filaments. The Well release contains 256×256 fields of particle concentration, fluid velocity, the second moment of the particle orientation distribution, and the strain rate at 81 times, for a range of activity and alignment strengths (Ohana et al., 2024). These fields are low-order moments of the orientation distribution that the source simulator actually integrates, and the equation for the second moment involves the fourth moment, which is not released. No closed equation for the released fields exists, so any successful program is an effective model with an implicit closure. In the original study, this is addressed using learned closures (Maddu et al., 2024).

A.3. Measured non-reciprocal active droplets

The final task uses a binary oil-droplet system filmed over multiple trajectories using microscopy (Liu et al., 2024a). Bromooctane (BrOct) and ethoxynonafluorobutane (EFB) droplets are dispersed in aqueous Triton X-100. The surfactant preferentially solubilizes BrOct, and oil-filled micelles transfer it to the EFB droplets; the resulting interfacial-tension gradients and Marangoni flows make the interaction non-reciprocal. BrOct droplets repel each other and chase EFB droplets. Depending on droplet diameters and number ratios, the system forms moving chains, loops, compact clusters, and flower-like arrangements, which a minimal particle model with chemotactic attraction and steric repulsion reproduces only qualitatively (Liu et al., 2024b).

The exact experimental release is Zenodo record 11180025, licensed CC BY 4.0 (Liu et al., 2024a), spanning nominal diameter-ratio regimes 0.6, 1.0, and 2.0. It contains 25 retained movies in a complete-movie 13/6/6 train/validation/test split frozen from the documented initial number ratios. Every export is decoded onto a common isotropic 2048×2048 source grid at 3.26 μm per pixel before any crop is taken, and each example is a 512×512 crop of that field carrying one grayscale intensity channel at four-second cadence. A model observes 32 frames and predicts the following 16, a 64-second forecast. Windows start at prepared frames 0, 48, 96, and 144, and three evaluation crops are drawn per movie and window start,

which gives the 72 test windows. Evaluation uses that fixed centered 256×256 mask, a quarter of the crop area; its 128-pixel margin is what closes the 64-second forecast, because on training movies 0.998 of the tracked droplets scored at the terminal step were already inside the crop at the last observed frame. The single channel is a raw camera intensity and does not identify BrOct from EFB.

Appendix B. Oil droplet forecasts

Figure 5 shows the forecasts each method returns on the oil droplets at three horizons.

Appendix C. Program-level interpretation

C.1. Gray–Scott

The inspected selected program separates the Fourier-diagonal linear diffusion operator from the local nonlinear reaction. For each Fourier mode with linear operator L, ETDRK4 advances the semilinear system using $\exp(hL)$ and coefficient functions of hL (Cox and Matthews, 2002). Periodic derivatives therefore require FFTs rather than boundary stencils, while AB^2, $f(1-A)$, and $(f+k)B$ are evaluated pointwise. The program also uses observed frames to initialize its spectral state before the forecast. This combination matches the structure of the data generator and explains why the result should be described as reconstruction under context.

The implementation audit found that a later selected variant changed both coefficient evaluation and warm-start logic. Kassam and Trefethen showed that contour-based coefficient evaluation can improve numerical robustness near small eigenvalues (Kassam and Trefethen, 2005); analytic Cox–Matthews formulas can also be stable when implemented with appropriate small-argument branches. The benchmark does not contain a one-change-at-a-time ablation, so we report the code difference without assigning causality.

C.2. Active matter

The underlying generator evolves $\Psi(x, y, \theta, t)$ with Fourier pseudospectral operations and a semi-implicit scheme, then writes low-order moments. A moment-level predictor receives c, $\mathbf{D}$, $\mathbf{u}$, and $\mathbf{E}$, but not the fourth moment $\mathbf{S}$ needed to close the $\mathbf{D}$ equation. Selected programs use combinations of periodic Stokes projection, transport, diffusion, algebraic consistency projections, and fitted or spectral per-mode propagators. Those ingredients are scientifically recognizable, yet none makes the missing state identifiable. We therefore avoid equating lower vRMSE with recovery of the source kinetic model.

Appendix D. Hyperparameters

Table 2 and Table 3 list the settings of every method in Table 1, for each dataset where they differ. AutoSim is run four times on Gray–Scott and active matter and three times on the oil droplets, and every baseline is run at three seeds on all three datasets, except SymTorch on Gray–Scott, which is reported at two after one seed was excluded as a preliminary run.

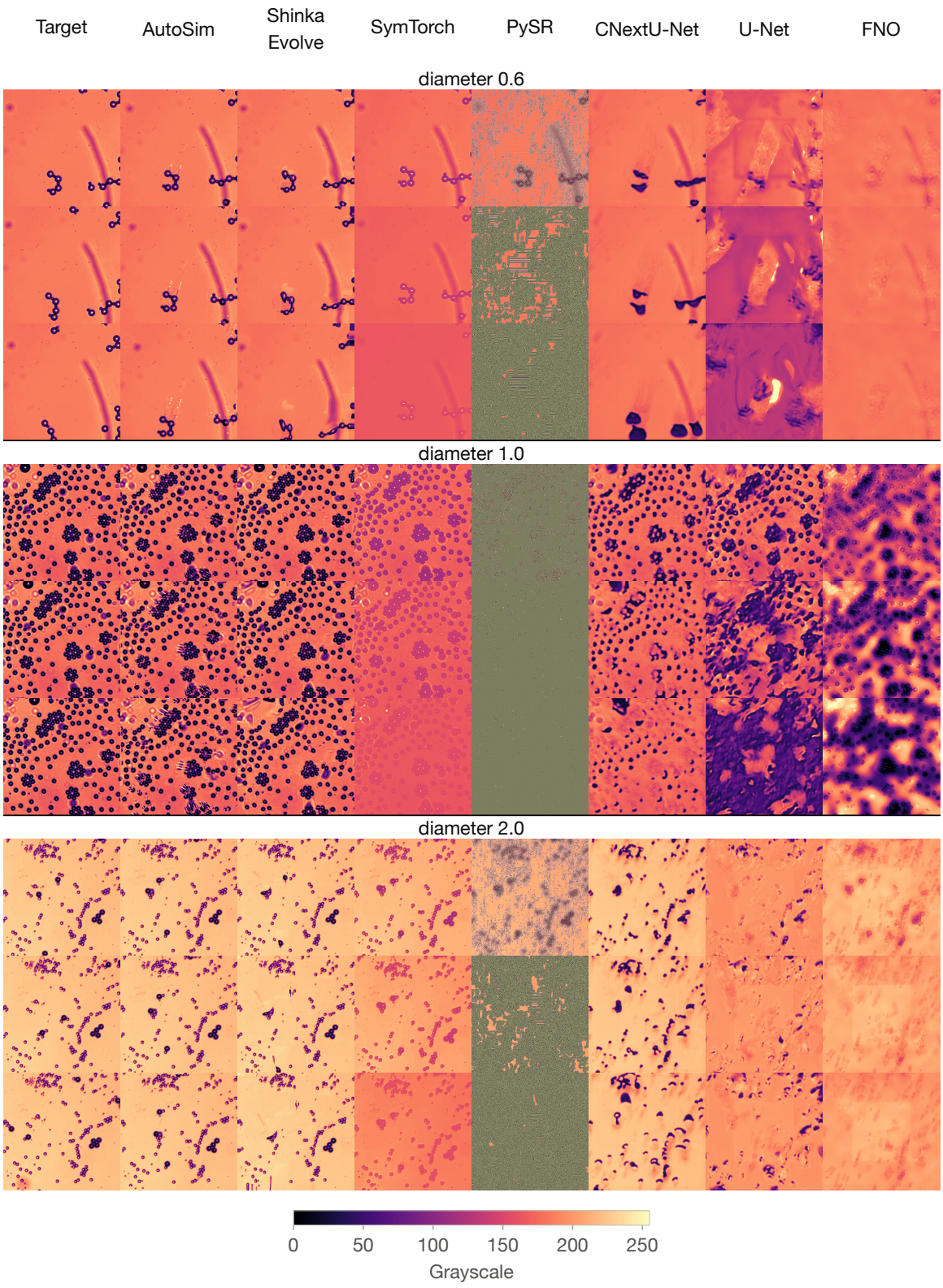


Figure 5: Predicted grayscale fields on three test movies, one per diameter-ratio regime; within each group the rows are $t{+}4$, $t{+}8$ and $t{+}16$ after the last observed frame, and the group label is the nominal diameter ratio. The first column is the target; every panel in a row is drawn on the target's grayscale range. The AutoSim column is the reported program. DISCO was not run on this task.

Table 2: Search-method settings for AutoSim and ShinkaEvolve.

Setting	Gray–Scott	Active matter	Oil droplets
AutoSim			
Coding agent model	`fugu-ultra`	`fugu-ultra`	`gpt-5.6-sol`
Reasoning effort	max		
Search budget	30 iterations		
Independent runs	4	4	3
Candidate language	Julia		
Selection	selection agent, validation RMSE objective		
Evaluation window	32 steps	32 steps	16 steps
Stop criterion	per-channel validation RMSE $\leq 10^{-4}$		none
ShinkaEvolve			
LLM	`fugu`		
Generations	80	80	30
Islands, archive size	5, 20		
Sampling temperatures	0, 0.5, 1.0		
Tokens per proposal	8192		
Patch mix	0.85 diff, 0.10 full, 0.05 crossover		
Fitness	RMSE over an 8-step rollout		
Independent runs	3		

Appendix E. AI and LLM use

LLMs are part of the studied method: coding-agent calls select, write, and interpret candidate programs. They were also used to help draft manuscript text and plotting code. All numerical values are read from trusted evaluator artifacts or deterministic plotting data, and the authors remain responsible for checking the resulting code, claims, citations, and presentation.

SIMBIOCHEM Submission Checklist

1. **Claims**

   Question: Do the main claims in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The paper separates forecast performance from mechanistic interpretation and distinguishes reconstruction of Gray–Scott, closure of latent active-matter moments, and observation-level modeling of measured oil droplets.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work?

   Answer: [Yes]

Justification: section 6 covers supplied scientific context, reference-seed coupling, unpaired method cohorts, unequal budgets, active-matter non-closure, optical-density uncertainty, finite field of view, and the limits of field error for droplet organization.

3. **AI / LLM disclosure** *(important)*

   Question: Does the paper disclose any use of AI/LLMs in the research or writing (e.g., ideation, code generation, data processing, drafting), in line with the NeurIPS LLM policy?

   Answer: [Yes]

   Justification: LLM coding agents are part of the studied method and assisted with manuscript and plotting-code drafting. The scope and author responsibility are disclosed in the appendix.

4. **Research integrity & originality**

   Question: Do you confirm that this is an original submission—not a duplicate or lightly tweaked resubmission, nor one of multiple versions of the same work submitted to this workshop—and that you comply with research-integrity guidelines?

   Answer: [Yes]

   Justification: This is the only version submitted to SIMBIOCHEM.

5. **Reproducibility** *(optional)*

   Question: Where applicable, does the paper provide enough information (data, code, settings) to reproduce the main results?

   Answer: [Yes]

   Justification: Public dataset identifiers, frozen simulation cells, deterministic plotting data, the AutoSim interface, and the complete active-droplet preprocessing, split, metadata, and scoring contracts are specified.

Table 3: Symbolic and neural baseline settings.

Method	Setting	Gray–Scott	Active matter	Oil droplets
SymTorch	Predictor	per-cell MLP, hidden width 32		
	Optimizer	Adam, lr 10^{-3}, no schedule		
	Epochs, batch size	20, 4	20, 4	20, 2
	Batches per epoch	200		
	Transitions per epoch	all	all	600
	Input steps	4	4	32
	Loss	MSE over a 4-step rollout		
	Distillation	PySR each epoch, 20 iterations		
PySR	Generations	200		
	Populations	15	15	8
	Pixels sampled	300	300	512
	Max complexity	25		
	Operators	$+$, $-$, $\times$, $\div$, negation, square		
	Fitness	RMSE over an 8-step rollout, 5 trajectories		
	Early stopping	none		
DISCO	Architecture	width 384, patch 16, 12 blocks, 6 heads		
	Pretraining	50 epochs, six The Well datasets		—
	Optimizer	lr 10^{-4}, decay 10^{-3}, cosine		—
	Batch size	8, accumulated over 5 steps		—
	Input steps	4	4	—
	Integration	adaptive, 32 steps, rtol 5×10^{-6}		—
CNextU-Net	Architecture	42 initial features, 4 stages, 2 blocks per stage		
	Optimizer	AdamW, lr 10^{-4}, decay 10^{-4}, cosine		
	Epochs, batch size	50, 12	50, 12	50, 2
	Batches per epoch	200		
	Transitions per epoch	all	all	600
	Input, output steps	4, 1	4, 1	32, 1
	Loss	single-step MSE on normalized fields		
	Early stopping	patience 5	patience 5	none
U-Net	Architecture	32 initial features, 4 pooling stages		
	Optimizer	Adam, lr 10^{-4}, cosine		
	Epochs, batch size	50, 12	50, 12	50, 2
	Batches per epoch	200		
	Transitions per epoch	all	all	600
	Input, output steps	4, 1	4, 1	32, 1
	Loss	single-step MSE on normalized fields		
	Early stopping	patience 5	patience 5	none
FNO	Architecture	16 Fourier modes per axis, 64 hidden channels		
	Optimizer	Adam, lr 10^{-4}, cosine		
	Epochs, batch size	40, 32	40, 32	40, 2
	Batches per epoch	200		
	Transitions per epoch	all	all	600
	Input, output steps	4, 1	4, 1	32, 1
	Loss	single-step MSE on normalized fields		
	Early stopping	patience 5	patience 5	none
Persistence	repeats the last observed frame; no fitted parameters			